

Documento de Requisitos

Trabalho Final — Escolha de Comandante em uma Frota de Drones

Algoritmos: Paxos e Lamport-Shostak-Pease / OM(m)

Ambiente: Python e Mininet

1. Identificação do Projeto

Nome do projeto: Escolha de Comandante em uma Frota de Drones

Disciplina: Sistemas Distribuídos

Linguagem escolhida: Python

Algoritmos escolhidos: Paxos e Lamport-Shostak-Pease / OM(m)

Ambiente de execução: modo local em Python e modo emulado com Mininet

Tipo de trabalho: trabalho prático em grupo

Tamanho do grupo: 2 ou 3 alunos

Objetivo geral:

Desenvolver um software capaz de simular uma frota de drones que precisa escolher um comandante por meio de algoritmos de consenso, mesmo diante de falhas de comunicação, falhas de nós e falhas bizantinas. O projeto também deve possuir uma camada de emulação com Mininet, em que os drones são representados como hosts emulados e se comunicam por sockets TCP usando mensagens em JSON.

2. Contextualização do Problema

O trabalho propõe um cenário em que uma frota de drones autônomos precisa escolher um comandante para coordenar uma missão de reconhecimento. Como os drones se comunicam em rede, podem ocorrer falhas de comunicação, atrasos, perda de mensagens ou comportamentos contraditórios.

Cada drone será tratado como um nó de um sistema distribuído. Esses drones precisam trocar mensagens e chegar a uma decisão comum sobre quem será o comandante da frota.

O desafio principal é garantir que os drones corretos concordem com o mesmo comandante, mesmo quando alguns drones falham, deixam de responder, atrasam mensagens ou enviam informações incorretas.

Para atender ao enunciado, o projeto possui duas formas de execução:

1. **Execução local:** usada para validar todos os cenários, algoritmos, métricas, CSV, gráficos, relatório e testes automatizados.
 2. **Execução em Mininet:** usada para representar os drones como hosts emulados, conectados por uma rede virtual, comunicando-se por sockets TCP e mensagens JSON.
-

3. Justificativa da Escolha dos Algoritmos

Foram escolhidos dois algoritmos de consenso:

- **Paxos**, para cenários com falhas não bizantinas;
- **Lamport-Shostak-Pease / OM(m)**, para cenários com falhas bizantinas.

Essa escolha foi feita porque o trabalho envolve diferentes tipos de falha. O Paxos é adequado para lidar com falhas como crash, omissão e temporização, nas quais os nós podem parar, atrasar ou perder mensagens, mas não tentam enganar os demais. Já o Lamport-Shostak-Pease é adequado para cenários bizantinos, nos quais alguns participantes podem enviar valores incorretos ou contraditórios.

No projeto, o valor decidido pelos algoritmos representa o **comandante da frota de drones**.

3.1 Por que usar Paxos?

O Paxos foi escolhido porque é um algoritmo clássico de consenso em sistemas distribuídos e utiliza o conceito de maioria para garantir que os nós corretos cheguem a um único valor. Ele é apropriado quando as falhas são não bizantinas, ou seja, quando os drones podem parar de responder, omitir mensagens ou sofrer atrasos, mas não enviam informações maliciosamente contraditórias.

No projeto, foi implementado um **Paxos de valor único**, voltado para a escolha de um comandante. A implementação considera propostas numeradas, mensagens **PREPARE**, **PROMISE**, **ACCEPT_REQUEST**, **ACCEPTED** e **DECIDE**, além da preservação de valores previamente aceitos.

3.2 Por que usar Lamport-Shostak-Pease / OM(m)?

O algoritmo Lamport-Shostak-Pease está diretamente relacionado ao problema clássico dos generais bizantinos. Ele foi escolhido porque o enunciado envolve drones que podem enviar informações incorretas ou contraditórias.

No projeto, foi usada uma versão baseada em **OM(m)**, em que **m** representa a quantidade de falhas bizantinas toleradas. O algoritmo trabalha com envio de valores, repasses recursivos e decisão por maioria, considerando apenas os drones corretos na verificação de acordo.

4. Objetivos do Sistema

O sistema deverá permitir a simulação de uma frota de drones executando algoritmos de consenso para escolher um comandante.

Os principais objetivos são:

- criar drones como nós de um sistema distribuído;
- permitir a escolha de um comandante;
- implementar Paxos para falhas não bizantinas;
- implementar Lamport-Shostak-Pease / OM(m) para falhas bizantinas;
- simular falhas crash, omissão e temporização;
- simular falhas bizantinas;
- verificar acordo, validade e terminação;
- medir mensagens enviadas, recebidas, omitidas, atrasadas e contraditórias;
- medir tempo de execução;
- executar cenários com 5 e 10 drones;
- gerar CSV com os resultados;
- gerar gráficos simples;
- gerar relatório automático em TXT;
- possuir modo detalhado de execução com `--verbose`;
- possuir uma camada de emulação com Mininet;
- documentar claramente a execução local e a execução em ambiente emulado.

5. Linguagem e Tecnologias

5.1 Linguagem

O projeto será desenvolvido em **Python 3**.

5.2 Bibliotecas Utilizadas

Bibliotecas nativas:

- `socket`, para comunicação TCP na emulação;
- `threading`, para escuta de mensagens em paralelo;
- `time`, para controle de tempo, atrasos e timeouts;
- `random`, para simulação de falhas e repetibilidade com seed;
- `json`, para estruturação de mensagens e cenários;
- `argparse`, para argumentos de terminal;
- `csv`, para exportação dos resultados;
- `os` e `pathlib`, para manipulação de arquivos e pastas;

- `unittest`, para testes automatizados.

Biblioteca opcional:

- `matplotlib`, para geração de gráficos simples.

5.3 Ambiente de Emulação

O enunciado solicita o uso de um ambiente de emulação, como o **Mininet**. Por isso, o projeto possui uma pasta específica chamada `emulacao/`.

A emulação com Mininet tem o objetivo de representar cada drone como um host emulado. A comunicação entre o controlador e os drones ocorre por sockets TCP, com mensagens estruturadas em JSON.

A execução em Mininet deve ser feita em ambiente Linux, WSL compatível ou máquina virtual com Mininet e Open vSwitch instalados.

6. Escopo do Projeto

6.1 O que o sistema deve fazer

O sistema deve:

- criar uma frota com 5 ou 10 drones;
- permitir escolher o algoritmo de consenso;
- executar Paxos em cenários com falhas não bizantinas;
- executar Lamport-Shostak-Pease / OM(m) em cenários com falhas bizantinas;
- configurar o número de drones;
- configurar o tipo de falha;
- configurar a quantidade de drones falhos;
- configurar taxa de omissão;
- configurar atraso máximo;
- configurar seed;
- registrar mensagens enviadas e recebidas;
- registrar mensagens omitidas, atrasadas e contraditórias;
- verificar acordo, validade e terminação;
- executar cenários predefinidos em JSON;
- executar todos os cenários principais com `--todos`;
- exportar resultados em CSV;
- gerar gráficos;
- gerar relatório automático;
- executar em modo detalhado com `--verbose`;
- possuir scripts para execução em Mininet;
- salvar resultados na pasta `resultados/`.

6.2 O que o sistema não precisa fazer

O sistema não precisa:

- controlar drones reais;
 - usar sensores, GPS ou mapas reais;
 - ter interface gráfica;
 - usar banco de dados;
 - implementar criptografia real;
 - simular combate real;
 - executar em drones físicos;
 - implementar replicação de log no Paxos;
 - substituir completamente a versão local pela versão Mininet.
-

7. Modelagem do Sistema

7.1 Drone

Cada drone representa um nó do sistema distribuído.

Cada drone deve possuir:

- identificador único;
- candidato proposto;
- comportamento;
- estado de falha;
- mensagens recebidas;
- decisão final;
- contador de mensagens enviadas;
- contador de mensagens recebidas;
- contador de mensagens omitidas;
- contador de mensagens atrasadas;
- contador de mensagens contraditórias;
- informações de proposta prometida e aceita, no caso do Paxos;
- valores recebidos, no caso do OM(m).

Comportamentos possíveis:

- correto;
- crash;
- omissão;
- temporização;
- bizantino.

7.2 Comandante

O comandante é o drone escolhido pelo algoritmo de consenso.

Ele pode ser:

- o drone proposto inicialmente;
- o valor escolhido por maioria;
- o valor preservado pelo Paxos em caso de proposta previamente aceita;
- o valor resultante da maioria no OM(m).

7.3 Mensagens

As mensagens trocadas pelo sistema são padronizadas.

Exemplo geral de mensagem:

```
{  
  "tipo": "PREPARE",  
  "origem": 0,  
  "destino": 1,  
  "numero_proposta": 1,  
  "candidato": 2,  
  "algoritmo": "paxos"  
}
```

Tipos de mensagem para Paxos:

- **PREPARE;**
- **PROMISE;**
- **ACCEPT_REQUEST;**
- **ACCEPTED;**
- **DECIDE.**

Tipos de mensagem para Lamport-Shostak-Pease / OM(m):

- **ORDER;**
- **FORWARD;**
- **DECISION.**

Na emulação com Mininet, as mensagens são enviadas por sockets TCP em formato JSON.

8. Algoritmo 1 — Paxos

8.1 Finalidade

O Paxos será usado para escolher o comandante em cenários com falhas não bizantinas.

Ele será aplicado nos casos de:

- falha crash;
- falha de omissão;
- falha de temporização.

8.2 Papéis do Paxos

A implementação considera os seguintes papéis:

- **Proposer:** drone que propõe um comandante;
- **Acceptor:** drone que promete e aceita propostas;
- **Learner:** drone que aprende a decisão final.

Para simplificar a estrutura, um mesmo drone pode exercer mais de um papel.

8.3 Funcionamento Geral

O Paxos implementado é de **valor único**, voltado à escolha de um comandante.

Fluxo geral:

1. Um drone inicia uma proposta numerada.
2. O proposer envia **PREPARE** aos acceptors.
3. Os acceptors respondem com **PROMISE**, caso a proposta seja nova.
4. O **PROMISE** informa se o acceptor já havia aceitado algum valor antes.
5. Se houver maioria de **PROMISE**, o proposer escolhe o valor adequado.
6. Se algum valor já havia sido aceito, o proposer preserva o valor aceito de maior número.
7. Se nenhum valor anterior existir, o proposer usa seu candidato inicial.
8. O proposer envia **ACCEPT_REQUEST**.
9. Os acceptors respondem com **ACCEPTED**.
10. Se houver maioria de **ACCEPTED**, o comandante é decidido.
11. A decisão é divulgada aos drones corretos com **DECIDE**.

8.4 Regras do Paxos

O Paxos deverá obedecer às seguintes regras:

- cada proposta deve ter um número único;
- um acceptor guarda a maior proposta prometida;

- um acceptor não deve aceitar proposta antiga depois de prometer proposta maior;
- o **PROMISE** deve carregar o valor previamente aceito, se existir;
- o proposer deve preservar o valor aceito de maior número;
- a decisão só acontece com maioria;
- drones em crash não respondem;
- mensagens omitidas não são contabilizadas como recebidas;
- mensagens atrasadas são registradas;
- se não houver maioria, o sistema registra que não houve decisão.

8.5 Simplificação da Implementação

O projeto não implementa replicação de log. O objetivo é realizar consenso sobre um único valor: o comandante da frota.

Assim, o Paxos implementado é adequado ao problema do trabalho porque o sistema precisa decidir apenas um comandante por execução.

9. Algoritmo 2 — Lamport-Shostak-Pease / OM(m)

9.1 Finalidade

O Lamport-Shostak-Pease / OM(m) será usado nos cenários com falhas bizantinas.

Ele é adequado porque trata participantes que podem enviar valores diferentes para destinatários diferentes, tentando causar divergência entre os drones corretos.

9.2 Ideia Geral

A ideia do algoritmo é:

1. Um comandante inicial envia um valor aos demais.
2. Os drones corretos repassam fielmente o valor recebido.
3. Drones bizantinos podem enviar valores contraditórios.
4. O processo ocorre de forma recursiva conforme o número de falhas toleradas.
5. Os drones corretos reúnem os valores recebidos.
6. A decisão é feita por maioria.
7. Se houver ausência de valor ou empate, utiliza-se um valor padrão.

9.3 OM(m)

O valor **m** representa a quantidade de falhas bizantinas toleradas.

Regras gerais:

- se **m = 0**, o comandante envia diretamente o valor aos demais;

- se $m > 0$, os tenentes repassam os valores recebidos em chamadas recursivas $OM(m-1)$;
- cada caminho de mensagem é registrado para evitar ciclos;
- drones corretos repassam o valor sem alterar;
- drones bizantinos podem alterar o valor enviado;
- a decisão final é calculada por maioria;
- em caso de empate ou ausência, usa-se valor padrão.

9.4 Valor Padrão

O algoritmo deve possuir um valor padrão para casos em que:

- uma mensagem não é recebida;
- ocorre empate;
- não há maioria clara.

Esse valor evita que o algoritmo fique indefinidamente sem decisão.

9.5 Limite Teórico

Para tolerar f falhas bizantinas, usa-se a condição clássica:

$$n \geq 3f + 1$$

Onde:

- n é o número total de drones;
- f é a quantidade de drones bizantinos.

Exemplos:

- 5 drones e 1 bizantino: dentro do limite;
- 5 drones e 2 bizantinos: fora do limite;
- 10 drones e 3 bizantinos: dentro do limite;
- 10 drones e 4 bizantinos: fora do limite.

Os cenários fora do limite devem ser executados para análise, mas o sistema deve avisar que não há garantia teórica clássica.

10. Tipos de Falhas

10.1 Falha Crash

Um drone para de funcionar.

Comportamento esperado:

- não envia mensagens;
- não responde mensagens;
- não decide comandante;
- os demais drones tentam continuar.

10.2 Falha de Omissão

Um drone deixa de enviar ou receber algumas mensagens.

Comportamento esperado:

- algumas mensagens são descartadas;
- o drone pode continuar ativo;
- o consenso pode ocorrer se ainda houver mensagens suficientes.

10.3 Falha de Temporização

Um drone demora para responder.

Comportamento esperado:

- mensagens podem ser entregues com atraso;
- o sistema registra mensagens atrasadas;
- o algoritmo pode decidir se ainda houver maioria suficiente.

10.4 Falha Bizantina

Um drone envia informações incorretas ou contraditórias.

Comportamento esperado:

- pode enviar um comandante diferente para cada drone;
- pode alterar valores recebidos;
- pode tentar gerar divergência;
- deve ser tratado pelo Lamport-Shostak-Pease / OM(m).

11. Requisitos de Consenso

11.1 Validade

Se todos os drones corretos propuserem ou receberem o mesmo valor inicial em um cenário válido, esse valor deve ser escolhido.

No Paxos, a validade também deve respeitar a regra de preservação de valor previamente aceito.

11.2 Acordo

Nenhum drone correto deve decidir um comandante diferente de outro drone correto.

11.3 Terminação

Todos os drones corretos devem chegar a uma decisão em tempo finito, desde que o cenário permita consenso dentro das condições do algoritmo.

11.4 Tolerância a Falhas

O sistema deve continuar funcionando mesmo com drones falhos, respeitando os limites do algoritmo utilizado.

12. Requisitos Funcionais

RF01 — Criar frota de drones

O sistema deve permitir criar uma frota com 5 ou 10 drones.

Cada drone deve possuir:

- identificador;
- comportamento;
- candidato proposto;
- decisão final;
- métricas de mensagens.

RF02 — Configurar algoritmo

O usuário deve conseguir escolher o algoritmo a ser executado.

Exemplos:

```
python main.py --algoritmo paxos --drones 5 --falha crash --falhos 1
```

```
python main.py --algoritmo lsp --drones 5 --falha bizantina --falhos 1
```

RF03 — Implementar Paxos

O sistema deve implementar Paxos para consenso de valor único.

O Paxos deve conter:

- **PREPARE;**

- PROMISE;
- ACCEPT_REQUEST;
- ACCEPTED;
- DECIDE;
- propostas numeradas;
- preservação de valor aceito;
- decisão por maioria.

RF04 — Implementar Lamport-Shostak-Pease / OM(m)

O sistema deve implementar OM(m) para cenários bizantinos.

O algoritmo deve conter:

- proposta inicial;
- repasses recursivos;
- registro de caminhos;
- valor padrão;
- decisão por maioria;
- tratamento de mensagens contraditórias.

RF05 — Simular falha crash

O sistema deve permitir selecionar drones que irão falhar por crash.

Esses drones não devem responder durante a simulação.

RF06 — Simular falha de omissão

O sistema deve permitir omitir mensagens.

Exemplo:

```
python main.py --algoritmo paxos --drones 5 --falha omissao --falhos 2 --taxa-omissao 0.3
```

RF07 — Simular falha de temporização

O sistema deve permitir atrasar mensagens.

Exemplo:

```
python main.py --algoritmo paxos --drones 5 --falha temporizacao --falhos 2 --atraso-max 3
```

RF08 — Simular falha bizantina

O sistema deve permitir que drones bizantinos enviem mensagens contraditórias.

Exemplo:

```
python main.py --algoritmo lsp --drones 5 --falha bizantina --falhos 1
```

RF09 — Registrar mensagens

O sistema deve registrar:

- mensagens enviadas;
- mensagens recebidas;
- mensagens omitidas;
- mensagens atrasadas;
- mensagens contraditórias;
- decisões tomadas.

RF10 — Verificar acordo

O sistema deve verificar se todos os drones corretos decidiram o mesmo comandante.

RF11 — Verificar validade

O sistema deve verificar se a decisão respeitou a proposta comum ou as regras do algoritmo utilizado.

RF12 — Verificar terminação

O sistema deve verificar se todos os drones corretos chegaram a uma decisão.

RF13 — Gerar métricas

O sistema deve calcular:

- tempo de execução;
- mensagens enviadas;
- mensagens recebidas;
- mensagens omitidas;
- mensagens atrasadas;
- mensagens contraditórias;
- comandante escolhido;
- acordo;
- validade;
- terminação.

RF14 — Executar todos os cenários

O sistema deve possuir modo para executar todos os cenários principais.

Exemplo:

```
python main.py --todos --seed 42
```

RF15 — Exportar CSV

O sistema deve gerar arquivos CSV com os resultados.

Exemplo:

```
python main.py --todos --seed 42 --csv
```

RF16 — Gerar gráficos

O sistema deve gerar gráficos simples a partir dos resultados.

Exemplo:

```
python main.py --todos --seed 42 --csv --graficos
```

RF17 — Gerar relatório automático

O sistema deve gerar relatório TXT.

Exemplo:

```
python main.py --todos --seed 42 --csv --graficos --relatorio
```

RF18 — Modo verbose

O sistema deve permitir execução detalhada.

Exemplo:

```
python main.py --algoritmo paxos --drones 5 --falha crash --falhos 1 --seed 42 --verbose
```

RF19 — Execução por cenário JSON

O sistema deve permitir executar cenários por arquivo JSON.

Exemplo:

```
python main.py --cenario cenarios/paxos_5_crash.json
```

RF20 — Execução em Mininet

O sistema deve possuir scripts para execução em Mininet.

Exemplo:

```
sudo python3 emulacao/mininet_topologia.py --algoritmo paxos --drones 5 --falha crash  
--falhos 1 --seed 42
```

RF21 — Verificação de ambiente Mininet

O sistema deve possuir script para verificar se o ambiente possui Python, Mininet e Open vSwitch.

Exemplo:

```
python3 emulacao/verificar_ambiente.py
```

13. Cenários Obrigatórios

13.1 Paxos com 5 drones

Configuração:

- algoritmo: Paxos;
- quantidade de drones: 5;
- falhas: crash, omissão e temporização;
- quantidade de falhos: 1 a 3.

Objetivo:

- verificar se o Paxos consegue escolher um comandante com falhas não bizantinas.

13.2 Paxos com 10 drones

Configuração:

- algoritmo: Paxos;
- quantidade de drones: 10;
- falhas: crash, omissão e temporização;
- quantidade de falhos: 1 a 6.

Objetivo:

- verificar o comportamento do Paxos em uma frota maior.

Observação:

- se a quantidade de falhos impedir maioria, o sistema deve registrar que não houve decisão.

13.3 LSP / OM(m) com 5 drones

Configuração:

- algoritmo: Lamport-Shostak-Pease / OM(m);
- quantidade de drones: 5;
- falha: bizantina;
- quantidade de falhos: 1 a 2.

Objetivo:

- testar consenso em presença de mensagens contraditórias.

Observação:

- com 1 falha bizantina, o cenário está dentro do limite;
- com 2 falhas bizantinas, o cenário está fora do limite teórico clássico.

13.4 LSP / OM(m) com 10 drones

Configuração:

- algoritmo: Lamport-Shostak-Pease / OM(m);
- quantidade de drones: 10;
- falha: bizantina;
- quantidade de falhos: 1 a 4.

Objetivo:

- testar consenso bizantino com uma frota maior.

Observação:

- com até 3 falhas bizantinas, o cenário está dentro do limite;
- com 4 falhas bizantinas, o cenário está fora do limite teórico clássico.

14. Ambiente de Emulação com Mininet

14.1 Finalidade

A emulação com Mininet foi adicionada para atender ao requisito do enunciado de usar um ambiente de emulação.

Na emulação:

- cada drone é representado por um host Mininet;
- existe um host controlador;
- todos são conectados a um switch;
- os drones executam servidores TCP;
- as mensagens são enviadas em JSON;
- os resultados são salvos na pasta `resultados/`.

14.2 Estrutura da Topologia

Topologia sugerida:

d0 \

d1 \

d2 --- s1 --- coord

d3 /

d4 /

Onde:

- `d0`, `d1`, `d2`, `d3`, `d4` são drones;
- `s1` é o switch;
- `coord` é o host controlador.

14.3 Arquivos da Emulação

Estrutura:

emulacao/

├── __init__.py

├── drone_socket.py

- |— controlador_emulacao.py
- |— mininet_topologia.py
- |— verificar_ambiente.py
- |— README.md
- └— README_MININET_INICIANTE.md

14.4 Comandos da Emulação

Verificar ambiente:

```
python3 emulacao/verificar_ambiente.py
```

Limpar redes antigas do Mininet:

```
sudo mn -c
```

Executar Paxos em Mininet:

```
sudo python3 emulacao/mininet_topologia.py --algoritmo paxos --drones 5 --falha crash  
--falhos 1 --seed 42
```

Executar LSP / OM(m) em Mininet:

```
sudo python3 emulacao/mininet_topologia.py --algoritmo lsp --drones 5 --falha bizantina  
--falhos 1 --seed 42
```

Executar por cenário:

```
sudo python3 emulacao/mininet_topologia.py --cenario cenarios/paxos_5_crash.json
```

```
sudo python3 emulacao/mininet_topologia.py --cenario cenarios/lsp_5_bizantina_1.json
```

14.5 Observação sobre Validação

A versão Mininet deve ser validada em uma VM Linux, WSL compatível ou ambiente Linux com Mininet e Open vSwitch instalados.

A ausência de Mininet não deve impedir o funcionamento da versão local do projeto.

15. Requisitos Não Funcionais

RNF01 — Organização

O projeto deve ser organizado em módulos.

Estrutura esperada:

drones-consenso/

├─ main.py

├─ drone.py

├─ mensagem.py

├─ rede.py

├─ falhas.py

├─ paxos.py

├─ lsp.py

├─ simulador.py

├─ metricas.py

├─ exportador.py

├─ graficos.py

├─ relatorio.py

├─ log_simulacao.py

├─ cenarios/

├─ resultados/

├─ emulacao/

├─ tests/

|— requirements.txt

|— README.md

RNF02 — Clareza

O código deve ser simples, com comentários curtos e didáticos.

RNF03 — Portabilidade

A versão local deve funcionar em Windows, Linux ou macOS com Python instalado.

A versão Mininet deve funcionar em Linux ou VM Linux com Mininet e Open vSwitch.

RNF04 — Reprodutibilidade

O sistema deve aceitar seed.

Exemplo:

```
python main.py --algoritmo paxos --drones 5 --falha crash --falhos 1 --seed 42
```

RNF05 — Tempo de execução

Os cenários devem terminar em tempo aceitável.

Observação:

- OM(m) pode gerar muitas mensagens em cenários maiores, principalmente com 10 drones e 4 bizantinos;
- o modo verbose deve ser usado preferencialmente em cenários pequenos.

RNF06 — Tratamento de erros

O sistema deve exibir mensagens claras quando:

- algoritmo for inválido;
- tipo de falha for inválido;
- combinação algoritmo/falha for incompatível;
- quantidade de drones for inválida;
- quantidade de falhos for incompatível;
- não houver maioria;
- cenário estiver fora do limite teórico;
- Mininet não estiver instalado;
- matplotlib não estiver disponível.

RNF07 — Testabilidade

O sistema deve possuir testes para:

- drones;
 - mensagens;
 - falhas;
 - Paxos;
 - LSP / OM(m);
 - métricas;
 - cenários JSON;
 - CSV;
 - gráficos;
 - relatório;
 - verbose;
 - emulação por sockets;
 - importação segura sem Mininet.
-

16. Métricas de Avaliação

O sistema deve coletar:

- algoritmo utilizado;
 - cenário executado;
 - quantidade de drones;
 - tipo de falha;
 - quantidade de falhos;
 - comandante escolhido;
 - decisão alcançada;
 - tempo de execução;
 - mensagens enviadas;
 - mensagens recebidas;
 - mensagens omitidas;
 - mensagens atrasadas;
 - mensagens contraditórias;
 - acordo;
 - validade;
 - terminação;
 - status do limite teórico;
 - observações sobre falhas ou ausência de garantia.
-

17. Arquivos de Configuração dos Cenários

O sistema deve usar arquivos JSON para definir cenários.

Exemplo Paxos:

```
{  
  "nome": "paxos_5_crash",  
  "algoritmo": "paxos",  
  "drones": 5,  
  "falha": "crash",  
  "falhos": 1,  
  "seed": 42,  
  "taxa_omissao": 0.0,  
  "atraso_max": 0  
}
```

Exemplo LSP / OM(m):

```
{  
  "nome": "lsp_5_bizantina_1",  
  "algoritmo": "lsp",  
  "drones": 5,  
  "falha": "bizantina",  
  "falhos": 1,  
  "seed": 42,  
  "taxa_omissao": 0.0,  
  "atraso_max": 0,  
  "nivel_om": 1,  
  "valor_padrao": 0  
}
```

18. Comandos Esperados

Executar Paxos com crash:

```
python main.py --algoritmo paxos --drones 5 --falha crash --falhos 1 --seed 42
```

Executar Paxos com omissão:

```
python main.py --algoritmo paxos --drones 5 --falha omissao --falhos 2 --taxa-omissao 0.3  
--seed 42
```

Executar Paxos com temporização:

```
python main.py --algoritmo paxos --drones 5 --falha temporizacao --falhos 2 --atraso-max 3  
--seed 42
```

Executar LSP / OM(m) com falha bizantina:

```
python main.py --algoritmo lsp --drones 5 --falha bizantina --falhos 1 --seed 42
```

Executar por cenário:

```
python main.py --cenario cenarios/paxos_5_crash.json
```

```
python main.py --cenario cenarios/lsp_5_bizantina_1.json
```

Executar todos os cenários:

```
python main.py --todos --seed 42
```

Executar todos os cenários com CSV, gráficos e relatório:

```
python main.py --todos --seed 42 --csv --graficos --relatorio
```

Executar com verbose em cenário pequeno:

```
python main.py --algoritmo paxos --drones 5 --falha crash --falhos 1 --seed 42 --verbose
```

```
python main.py --algoritmo lsp --drones 5 --falha bizantina --falhos 1 --seed 42 --verbose
```

Executar Mininet:

```
python3 emulacao/verificar_ambiente.py
```

```
sudo mn -c
```

```
sudo python3 emulacao/mininet_topologia.py --algoritmo paxos --drones 5 --falha crash  
--falhos 1 --seed 42
```

```
sudo python3 emulacao/mininet_topologia.py --algoritmo lsp --drones 5 --falha bizantina  
--falhos 1 --seed 42
```

19. Saída Esperada

19.1 Exemplo com Paxos

=====

Escolha de Comandante em Frota de Drones

Algoritmo: Paxos

Drones: 5

Falha: crash

Falhos: 1

Seed: 42

=====

Resultado do consenso:

Comandante escolhido: Drone 2

Decisão alcançada: Sim

Acordo: Sim

Validade: Sim

Terminação: Sim

Mensagens enviadas: 23

Mensagens recebidas: 20

Mensagens omitidas: 0

Mensagens atrasadas: 0

Mensagens contraditórias: 0

Tempo de execução: 0.000086 s

19.2 Exemplo com LSP / OM(m)

=====

Escolha de Comandante em Frota de Drones

Algoritmo: Lamport-Shostak-Pease / OM(m)

Drones: 5

Falha: bizantina

Falhos: 1

Seed: 42

=====

Resultado do consenso:

Comandante escolhido: Drone 2

Decisão alcançada: Sim

Acordo entre drones corretos: Sim

Validade: Sim

Terminação: Sim

Mensagens contraditórias: 4

Status do limite teórico: Dentro

Tempo de execução: 0.000120 s

19.3 Exemplo com Mininet

=====

EMULAÇÃO MININET - PAXOS

=====

Topologia:

coord + d0, d1, d2, d3, d4 conectados ao switch s1

Comunicação:

sockets TCP com mensagens JSON

Resultado:

Comandante escolhido: Drone 2

Decisão alcançada: Sim

Acordo: Sim

Terminação: Sim

Resultado salvo em:

resultados/emulacao_paxos_5_crash.txt

20. Critérios de Aceitação

O projeto será considerado adequado se:

- criar corretamente frotas com 5 e 10 drones;
 - permitir escolher algoritmo pelo terminal;
 - executar Paxos para falhas crash, omissão e temporização;
 - executar Lamport-Shostak-Pease / OM(m) para falhas bizantinas;
 - simular drones em crash;
 - simular mensagens omitidas;
 - simular mensagens atrasadas;
 - simular mensagens contraditórias;
 - verificar acordo, validade e terminação;
 - verificar o limite $n \geq 3f + 1$;
 - executar cenários obrigatórios;
 - gerar CSV;
 - gerar gráficos;
 - gerar relatório automático;
 - possuir modo verbose;
 - possuir testes automatizados;
 - possuir documentação clara;
 - possuir camada de emulação com Mininet;
 - explicar limitações e cenários fora do limite teórico.
-

21. Diferenciais Implementados ou Recomendados

21.1 Seed configurável

Permite repetir execuções.

```
python main.py --algoritmo paxos --drones 5 --falha crash --falhos 1 --seed 42
```

21.2 Modo **--todos**

Executa todos os cenários principais.

```
python main.py --todos --seed 42
```

21.3 Exportação CSV

Gera arquivo CSV com resultados.

```
python main.py --todos --seed 42 --csv
```

21.4 Relatório automático

Gera relatório TXT.

```
python main.py --todos --seed 42 --csv --relatorio
```

21.5 Gráficos

Gera gráficos PNG.

```
python main.py --todos --seed 42 --csv --graficos
```

21.6 Modo verbose

Mostra eventos internos.

```
python main.py --algoritmo paxos --drones 5 --falha crash --falhos 1 --seed 42 --verbose
```

21.7 Cenários JSON

Permite executar cenários por arquivo.

```
python main.py --cenario cenarios/paxos_5_crash.json
```

21.8 Mininet

Permite demonstrar drones como hosts emulados.

```
sudo python3 emulacao/mininet_topologia.py --algoritmo paxos --drones 5 --falha crash  
--falhos 1 --seed 42
```

21.9 Testes automatizados

Permite validar o projeto.

```
python -m unittest discover -s tests -v
```

22. Riscos do Projeto

22.1 Complexidade do Paxos

O Paxos pode ser difícil de implementar e explicar.

Mitigação:

- usar consenso de valor único;
- explicar as fases;
- preservar valores previamente aceitos;
- usar testes para propostas sucessivas.

22.2 Crescimento de mensagens no OM(m)

O OM(m) pode gerar muitas mensagens, principalmente com mais drones e mais falhas bizantinas.

Mitigação:

- registrar quantidade de mensagens;
- usar timeout;
- evitar verbose em cenários muito grandes;
- demonstrar verbose apenas em cenários pequenos.

22.3 Cenários fora do limite bizantino

Alguns cenários pedidos podem estar fora do limite clássico.

Mitigação:

- executar mesmo assim;
- avisar que não há garantia teórica;
- explicar no relatório e apresentação.

22.4 Ambiente Mininet

Mininet pode exigir configuração adicional em Linux ou VM.

Mitigação:

- manter versão local funcionando;
- criar guia de execução para iniciantes;
- criar script de verificação de ambiente;
- documentar erros comuns.

22.5 Dependência do matplotlib

A geração de gráficos depende do matplotlib.

Mitigação:

- tratar ausência da biblioteca com mensagem amigável;
 - manter CSV e relatório funcionando mesmo sem gráficos.
-

23. Plano de Desenvolvimento

Etapa 1 — Estrutura inicial

Criar pastas e arquivos principais.

Etapa 2 — Classes base

Criar *Drone*, *Mensagem* e estruturas auxiliares.

Etapa 3 — Comunicação local

Implementar envio e recebimento de mensagens no modo local.

Etapa 4 — Falhas

Implementar crash, omissão, temporização e bizantina.

Etapa 5 — Paxos

Implementar Paxos de valor único.

Etapa 6 — Paxos reforçado

Adicionar propostas numeradas, valores aceitos e preservação de segurança entre propostas.

Etapa 7 — LSP / OM(m)

Implementar Lamport-Shostak-Pease com OM(m), valor padrão e decisão por maioria.

Etapa 8 — Métricas

Implementar acordo, validade, terminação e contadores de mensagens.

Etapa 9 — Cenários JSON

Criar cenários predefinidos.

Etapa 10 — Modo `--todos`

Executar todos os cenários automaticamente.

Etapa 11 — CSV

Exportar resultados.

Etapa 12 — Gráficos

Gerar gráficos simples.

Etapa 13 — Relatório

Gerar relatório automático.

Etapa 14 — Verbose

Adicionar logs detalhados.

Etapa 15 — Emulação Mininet

Criar scripts para drones como hosts emulados.

Etapa 16 — README e documentação

Documentar execução local e Mininet.

Etapa 17 — Testes

Criar e executar testes automatizados.

Etapa 18 — Validação final

Executar testes, fluxo completo e validar Mininet na VM.

24. Entregáveis

O grupo deverá entregar:

- código-fonte em Python;
- arquivos JSON dos cenários;
- README principal;
- README da emulação;

- guia Mininet para iniciantes;
 - testes automatizados;
 - resultados em CSV;
 - gráficos PNG;
 - relatório automático em TXT;
 - arquivos de validação;
 - instruções de execução local;
 - instruções de execução em Mininet;
 - explicação dos algoritmos;
 - explicação das falhas;
 - análise dos resultados;
 - limitações do projeto.
-

25. Como Explicar na Apresentação

Na apresentação, o grupo deve explicar:

- o problema da escolha de comandante;
- o que representa cada drone;
- o que é uma falha crash;
- o que é omissão;
- o que é temporização;
- o que é falha bizantina;
- por que Paxos foi usado para falhas não bizantinas;
- por que Lamport-Shostak-Pease / OM(m) foi usado para falhas bizantinas;
- como o Paxos usa maioria;
- como o Paxos preserva valores previamente aceitos;
- como o OM(m) repassa valores recursivamente;
- o que significa $n \geq 3f + 1$;
- o que são acordo, validade e terminação;
- como os resultados são gerados;
- como funciona a emulação com Mininet.

Resumo sugerido:

O projeto implementa a escolha de comandante em uma frota de drones usando dois algoritmos de consenso. O Paxos foi usado para falhas não bizantinas, como crash, omissão e temporização, pois trabalha com propostas numeradas e decisão por maioria. Para falhas bizantinas, usamos Lamport-Shostak-Pease com OM(m), pois esse algoritmo trata participantes que podem enviar valores contraditórios. O sistema executa cenários com 5 e 10 drones, verifica acordo, validade e terminação, gera resultados em CSV, gráficos e relatório, e também possui uma camada de emulação com Mininet.

26. Conclusão dos Requisitos

O sistema proposto implementa uma simulação distribuída para escolha de comandante em uma frota de drones. O Paxos é utilizado para cenários com falhas não bizantinas, enquanto o Lamport-Shostak-Pease / OM(m) é utilizado para cenários com falhas bizantinas.

A implementação permite analisar o comportamento dos algoritmos em diferentes cenários de falha, verificando acordo, validade e terminação. Além disso, o projeto possui recursos de apoio, como execução automática de cenários, CSV, gráficos, relatório, modo verbose, testes automatizados e uma camada de emulação com Mininet.

A versão local é usada para validar todos os cenários e gerar resultados comparativos. A versão Mininet demonstra a execução em ambiente emulado, no qual os drones são representados como hosts separados e se comunicam por sockets TCP usando mensagens JSON.

Com isso, o projeto busca atender aos requisitos do trabalho de Sistemas Distribuídos e demonstrar, de forma prática e didática, conceitos importantes de consenso, tolerância a falhas e comunicação distribuída.